ISO/IEC JTC1 SC22 WG21 P2107R0
Author: Jens Maurer <Jens.Maurer@gmx.net>
Target audience: CWG, Plenary
2020-02-12

P2107R0: Core Issue 2436: US064 Copy semantics of coroutine parameters

This paper resolves C++17 NB ballot comment US064, also known as core issue 2436.

> Apply remaining coroutine TS issues to the working paper.
>
> Proposed change: Apply resolution of the issue 33. “Parameter copy wording does not capture the intent.” from p0664r8.

Wording

Change in 7.5.4.1 [expr.prim.id.unqual] paragraph 1:

> An identifier is only an id-expression if it has been suitably declared (Clause 9) or if it appears as part of a declarator-id (9.3). **An *identifier* that names a coroutine parameter refers to the copy of the parameter ([dcl.fct.def.coroutine]).** [Note: For operator-function-ids, see 12.6; for conversion-function-ids, see 11.4.7.2; for literal-operator-ids, see 12.6.8; for template-ids, see 13.3. A type-name or decltype-specifier prefixed by ~ denotes the destructor of the type so named; see 7.5.4.3. Within the definition of a non-static member function, an identifier that names a non-static member is transformed to a class member access expression (11.4.2). — end note]

Change in 9.4.4 [dcl.fct.def.coroutine] paragraph 13:

> When a coroutine is invoked, **after initializing its parameters ([expr.call]),** a copy is created for each coroutine parameter. ~~Each such~~ **For a parameter of type *cv* `T`, the** copy is ~~an object~~ **a variable of type *cv* `T`** with automatic storage duration that is direct-initialized ~~from an lvalue referring to the corresponding parameter if the parameter is an lvalue reference, and~~ from an xvalue **of type `T`** referring to ~~it otherwise~~ **the parameter**. **[Note: An original parameter object is never a const or volatile object (6.8.3 [basic.type.qualifier]).]** ~~A reference to a parameter in the function-body of the coroutine and in the call to the coroutine promise constructor is replaced by a reference to its copy.~~ The initialization and destruction of each parameter copy occurs in the context of the called coroutine. Initializations of parameter copies are sequenced before the call to the coroutine promise constructor and indeterminately sequenced with respect to each other. The lifetime of parameter copies ends immediately after the lifetime of the coroutine promise object ends. [Note: If a coroutine has a parameter passed by reference, resuming the coroutine after the lifetime of the entity referred to by that parameter has ended is likely to result in undefined behavior. — end note]